TALK N LOCK PVT. LTD.

AI/ML INTERN — TECHNICAL & BUSINESS ASSIGNMENT

AI-Powered Marketing Intelligence System

Option D — Content Recommendation

Project Report

Prepared by: Nikhil
Technology: Python, Pandas, Scikit-learn, LightGBM, Streamlit, Gemini API
Recommendation objective: Select the next content type using historical performance

Submission note

This report documents the prototype implementation described in the submitted source code. Model metric values are intentionally not fabricated; they are produced by train.py when the supplied dataset is generated and the model is trained.

1. Executive Summary

Talknlock operates across social media marketing, performance marketing, SEO, UGC, video production, and content marketing. The assignment asks for a practical AI/ML-powered Marketing Intelligence System that can reduce manual analysis and support measurable marketing decisions.

This project implements Option D — Content Recommendation. The system learns historical relationships between campaign context and a Performance_Score using a LightGBM regression model. At recommendation time, the system holds the campaign context constant and evaluates each available content type as a candidate. The candidate with the highest predicted performance score is recommended as the next content format.

A separate Gemini layer is used only after the ML recommendation is produced. Gemini converts the structured recommendation into an actionable content concept, including a title, concept, hook, and call to action. This separation keeps quantitative prediction inside the ML layer and uses the LLM for natural-language reasoning and content generation.

Component	Responsibility
Synthetic data generator	Creates 600 historical marketing records with realistic feature relationships and data-quality issues.
Preprocessing pipeline	Imputes missing values, encodes categorical variables, and scales numerical variables.
LightGBM model	Predicts Performance_Score for a candidate content configuration.
Recommendation engine	Scores all candidate content types and ranks them by predicted performance.
Streamlit prototype	Provides the employee-facing workflow and displays the recommendation.
Gemini AI layer	Generates content based on the ML-selected content type and campaign context.
MLOps module	Provides numerical/categorical drift checks and a Thompson Sampling A/B testing component.

2. Business Problem Definition

2.1 Current Problem

A marketing team can have many historical records but still require a human to inspect prior campaigns, compare platforms and formats, and decide what content should be produced next. This process becomes difficult to scale when multiple clients and campaigns are active simultaneously.

2.2 Proposed ML Decision

The prototype turns the decision into a ranking problem: for a given campaign context, predict the expected Performance_Score for each candidate content type and recommend the highest-scoring option.

2.3 Users

- Marketing managers deciding what content format to produce next.

- Content strategists planning campaign calendars.
- Account managers who need a data-backed recommendation for client discussions.
- Internal AI/ML teams monitoring recommendation quality and production data drift.

2.4 Why ML Is Appropriate

A simple historical average is a useful baseline, but it cannot condition the recommendation on the complete campaign context. A supervised ML model can learn interactions between industry, platform, topic, timing, reach, spend, and content type. The recommendation is therefore context-aware rather than a single global rule.

3. Dataset and Data Generation

Because production Talknlock data was not available, the prototype uses a synthetic dataset, which the assignment explicitly permits. The generator creates 600 records, satisfying the assignment requirement of at least 500 marketing/content records.

Feature	Type	Purpose
Client	Categorical	Represents different agency clients.
Industry	Categorical	Client/business context.
Platform	Categorical	Distribution channel such as Instagram or LinkedIn.
Content_Type	Categorical	Recommendation candidate: Reel, Carousel, Case Study, Infographic, or Blog Post.
Content_Topic	Categorical	Content theme such as Education, Trends, or Promo.
Posting_Day	Categorical	Day of publication.
Posting_Hour	Numerical	Hour of publication.
Reach	Numerical	Target/reported reach.
Ad_Spend	Numerical	Campaign spend.
Performance_Score	Target	Historical outcome used for supervised learning.

3.1 Synthetic Data Assumptions

The generator introduces controlled relationships so the ML problem is learnable. Instagram and LinkedIn receive different platform effects; content types receive different historical effects; topics and posting hours also contribute to the score. Random noise prevents the target from being a deterministic formula.

3.2 Data Quality Issues

- Approximately 10% of Content_Topic values are set to missing.
- Approximately 5% of Ad_Spend values are set to missing.
- Five Reach values are replaced with an extreme outlier of 9,999,999.
- The preprocessing pipeline is therefore required to handle missing and anomalous observations.

4. Exploratory Data Analysis

The primary analytical question is not simply which content type has the highest average score, but whether content performance varies meaningfully by platform, industry, topic, timing, and format. The training workflow therefore also generates a historical content-performance table and a visualization.

4.1 Historical Content Performance

Content Type	Expected analytical interpretation
Reel	Compare its historical average with other formats; it is intentionally given a positive synthetic effect.
Case Study	Represents a format with a strong business/education orientation.
Carousel	Represents a multi-slide educational/explanatory format.
Infographic	Represents visually structured informational content.
Blog Post	Represents long-form written content.

The generated file data/content_performance.csv contains the actual historical averages after execution. The corresponding data/content_performance.png provides a compact visual comparison suitable for the prototype.

4.2 Business Interpretation

Historical averages provide a useful baseline but can hide context. For example, a format may have a strong overall average because it is frequently used on a platform where it performs well. The ML approach attempts to condition the recommendation on the campaign attributes instead of assuming one format is universally best.

5. Machine Learning Approach

5.1 Problem Formulation

The model is formulated as supervised regression. The target is Performance_Score. Content_Type is retained as an input feature rather than being used as the label. This allows the trained model to estimate what would happen if a particular content type were selected.

At inference time, the system creates one candidate record for every content type while keeping the other campaign inputs constant. It predicts a score for every candidate and sorts the results in descending order.

5.2 Model Selection

LightGBM Regressor was selected because the problem contains a mixture of categorical and numerical predictors, non-linear relationships, and interactions. It is also computationally practical for a prototype and provides a strong tabular-data baseline without requiring a large deep-learning stack.

5.3 Preprocessing

Feature group	Processing	Reason
Categorical	Constant imputation + OneHotEncoder(handle_unknown='ignore')	Handles missing values and unseen categories at inference time.
Numerical	KNNImputer + RobustScaler	Handles missing numerical values and reduces sensitivity to the synthetic Reach outlier.
Pipeline	ColumnTransformer + sklearn Pipeline	Ensures identical preprocessing during training and prediction.

5.4 Train/Test Split

The current prototype uses an 80/20 train/test split with random_state=42. This makes the experiment reproducible. For a production system, a time-aware validation strategy would be preferable because future content recommendations should be evaluated against temporally later observations.

6. Model Evaluation and Baseline

The assignment asks for more than a single accuracy number and specifically calls for a baseline, model performance, overfitting/underfitting considerations, limitations, and potential leakage. This project uses regression metrics appropriate for a continuous Performance_Score.

Metric	Interpretation
R^2	Measures the proportion of target variance explained by the model relative to a mean baseline.
MAE	Average absolute prediction error; directly interpretable in Performance_Score units.
RMSE	Penalizes larger prediction errors more strongly than MAE.
Historical-average baseline	Provides a simple non-ML benchmark for content recommendation.

6.1 Baseline

The recommendation baseline is the content type with the highest historical mean Performance_Score. The training script also calculates a mean-target regression baseline on the test set. The ML system should be judged not only by its absolute metrics but by whether its context-aware recommendations improve on this simpler approach.

6.2 Metrics Output

The exact R^2, MAE, and RMSE values are generated by train.py from the deterministic synthetic dataset and should be copied from the terminal output after the final training run. They are deliberately not invented in this report.

Result	Value
R^2	Populate from final `python src/train.py` run
MAE	Populate from final `python src/train.py` run
RMSE	Populate from final `python src/train.py` run
Baseline MAE	Populate from final training/evaluation run
Baseline RMSE	Populate from final training/evaluation run

6.3 Leakage and Validation Risks

- Future information must not be used when generating a recommendation for an unpublished campaign.
- Production validation should use chronological splits rather than random splits when temporal effects are material.
- Client-level leakage should be considered if the same client's campaigns appear across both training and evaluation sets.
- Synthetic target-generation rules can make the benchmark easier than real-world data.

7. Recommendation Engine

The recommendation layer converts the regression model into a decision system. It is not a separate classifier. The same campaign context is duplicated across candidate rows, and Content_Type is changed for each candidate.

Candidate	Input change	Output
Reel	Content_Type = Reel	Predicted Performance_Score
Carousel	Content_Type = Carousel	Predicted Performance_Score
Case Study	Content_Type = Case Study	Predicted Performance_Score
Infographic	Content_Type = Infographic	Predicted Performance_Score
Blog Post	Content_Type = Blog Post	Predicted Performance_Score

The highest-scoring candidate surfaced as the recommended next content type. The Streamlit interface preserves the recommendation in session state so that clicking Generate AI Content does not reset the recommendation.

8. Explainability

The prototype prioritizes an understandable recommendation workflow. The employee sees the complete candidate ranking and the predicted score for each content type, which makes the recommendation auditable at the decision level.

A production version should add feature-level explanations using a method such as SHAP for the LightGBM model. This would allow a marketing manager to see whether platform, topic, posting time, reach, spend, or content format was driving a particular prediction.

The assignment explicitly allows feature importance, SHAP, LIME, partial dependence, or rule-based explanations. For this prototype, ranking visibility is implemented first; SHAP is a logical next enhancement.

9. AI / LLM Layer

Gemini is intentionally not responsible for the numerical recommendation. The ML model determines which content type has the highest expected performance. Gemini then converts that structured decision into useful marketing content.

Layer	Responsibility
Historical data	Provides evidence of past campaign performance.
ML model	Predicts Performance_Score for each candidate content type.
Recommendation engine	Ranks candidates and selects the highest-scoring option.
Gemini	Creates a title, concept, hook, and CTA constrained to the recommended format.
Human	Reviews and approves the final content before publication.

This architecture follows the assignment's principle that an LLM should not be added merely because it is fashionable. Structured prediction remains in traditional ML, while language generation and marketing ideation are handled by the LLM.

10. Working Prototype

The Streamlit application provides the following workflow:

1. Select Industry, Platform, and Topic.
2. Specify posting date/time, target reach, and ad spend.
3. Click Recommend Best Content.
4. The system scores all five candidate content types.
5. The highest-scoring format is shown as the recommendation.
6. The complete ranking and historical content-performance table are displayed.
7. Click Generate AI Content.
8. Gemini receives the stored ML recommendation and generates content specifically for that recommended format.

10.1 Generated Artifacts

File	Purpose
data/marketing_data.csv	Synthetic historical marketing dataset.
data/recommendation_model.pkl	Serialized preprocessing + LightGBM model and recommendation artifacts.
data/content_performance.csv	Historical average performance by content type.
data/content_performance.png	Visualization of historical content performance.

11. Production Architecture

A production implementation would extend the prototype into a monitored internal platform. The high-level flow is:

Stage	Production responsibility
Data collection	Social platforms, ad platforms, website analytics, CRM and content calendars.
Storage	Central analytical storage with client-level access controls.
Processing	Validation, feature engineering, missing-data handling and quality checks.
ML service	Versioned performance prediction and content-ranking model.
AI layer	Gemini or another approved LLM for explanations and content generation.
API/backend	Authentication, request validation, recommendation endpoints and audit logging.
Dashboard	Campaign inputs, ranked recommendations, explanations and feedback.
Monitoring	Data drift, prediction quality, latency, errors and business outcomes.
Retraining	Scheduled or trigger-based model updates using newly observed campaign outcomes.
Security	Client isolation, secrets management, least-privilege access and protection of campaign data.

12. MLOps and Monitoring

The mlops.py module includes Population Stability Index and Kolmogorov-Smirnov testing for numerical drift. It also includes categorical distribution monitoring and a Thompson Sampling A/B testing component.

12.1 Data Drift

Drift monitoring is important because the distribution of marketing inputs can change. For example, platform mix, reach levels, or ad-spend patterns may change over time. A production system should alert when input distributions move sufficiently far from the training distribution.

12.2 Recommendation A/B Testing

The Thompson Sampling component can support a comparison between human-selected recommendations and AI-selected recommendations. The eventual business metric should be an outcome such as engagement, conversion, or revenue rather than only offline prediction accuracy.

13. Business Impact

The business value is expected to come from reducing manual analysis and improving the consistency of content planning. Exact production impact cannot be claimed from the synthetic prototype; it must be measured after deployment.

Business metric	How it would be measured
Analyst hours saved	Time required to create a content recommendation before vs. after deployment.
Recommendation adoption	Percentage of recommendations accepted by marketers.
Content performance lift	Difference in engagement/performance between recommended and control content.
Conversion lift	Conversion or lead outcomes attributable to recommended content.
Client reporting efficiency	Reduction in manual analysis/report preparation time.
Revenue impact	Incremental revenue or qualified leads associated with improved content decisions.

A sensible rollout would begin with decision support rather than fully automated publishing. Human approval provides a safety and quality-control layer while sufficient real-world feedback is collected.

14. Limitations and Production Risks

- The dataset is synthetic and therefore cannot establish real-world marketing lift.
- The current feature set is smaller than the data available in a real marketing organization.
- The random train/test split is suitable for a prototype but should be replaced or supplemented by temporal validation.
- Historical bias can cause the recommender to reinforce existing content patterns.
- Performance_Score is a synthetic target and may not represent a single business KPI such as conversions or revenue.
- The current explainability layer is ranking-based; production use should add feature-level explanations.
- Model performance can degrade when audience behavior, platform algorithms, or campaign strategy changes.
- LLM-generated content still requires human review for brand safety, factual accuracy, and client requirements.

15. Conclusion

This prototype demonstrates a complete path from historical marketing data to an actionable content recommendation. The central design decision is to predict Performance_Score rather than directly predicting a content label. This makes the system capable of evaluating multiple candidate content types under the same campaign context.

The ML layer provides the quantitative decision, Streamlit provides the employee-facing workflow, Gemini turns the recommendation into usable content, and the MLOps module provides a foundation for monitoring and experimentation. The next major step is replacing synthetic data with real campaign outcomes and measuring whether the recommendations produce measurable business improvement.

Appendix A — Repository Structure

```
talknlock-marketing-ai/
├── src/
│   ├── app.py
│   ├── config.py
│   ├── generate.py
│   ├── mlops.py
│   ├── preprocess.py
│   └── train.py
├── data/
│   ├── marketing_data.csv
│   ├── recommendation_model.pkl
│   ├── content_performance.csv
│   └── content_performance.png
├── .env.example
├── .gitignore
├── README.md
└── requirements.txt
```

Appendix B — Execution

Generate the synthetic dataset:

```
python src/generate.py
```

Train the recommendation model:

```
python src/train.py
```

Run the Streamlit prototype:

```
streamlit run src/app.py
```